Challenge Question

Consider the following i/o loop:

```
void emit_sorted( unsigned S ) {
    int v;
    /** maybe some code here? */
    while( cin >> v ) {

       /** definitely some code here :) */

    }
    /** maybe some code here? */
}
```

Suppose the integers read from `cin` are in "mostly ascending order", no element is more than S locations away from its proper position if the sequence were wholly sorted. In other words, if $S = 0$, the inputs are already sorted; if $S = 1$ the input could be ordered with non-overlapping swaps between two sequential values.

Construct an algorithm that emits the sequence values in ascending order (ie `cout << x << endl;`) using **either** one stack **or** one queue (no other data structures allowed).

Can you prove your algorithm is as (space) efficient as possible?

(Obviously, using `std::sort( vector<int>::iterator, ...)` isn't part of an answer...)

Solution on next page...

```cpp
#include <iostream>
#include <queue>

using namespace std;
void emit_sorted( unsigned S )
{
    queue<int> Q;
    int v;
    while( cin >> v ) {
        /***
         * maintain the queue in ascending order of elements seen so far
         * essentially "insertion sort"
         */
        // wrap elements less than v to the back of the queue
        size_t i;
        for( i=0; i<Q.size() && Q.front()<v; ++i ) {
            Q.push( Q.front() );
            Q.pop();
        }
        // push v to the queue where it belongs in relation to the other
        // values
        Q.push(v);
        ++i;
        // finish wrapping elements >= v to the back of the queue
        for( ; i<Q.size(); ++i ) {
            Q.push( Q.front() );
            Q.pop();
        }
        // if we've seen at least S values, it is safe to emit the smallest
        // of them.
        if( Q.size() >= S ) {    // beware! == S doesn't work so well if S=0
            cout << Q.front() << endl;
            Q.pop();
        }
    }
    //  don't forget the S-1 elements still in the queue!
    while( !Q.empty() ) {
        cout << Q.front() << endl;
        Q.pop();
    }
}
```